Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

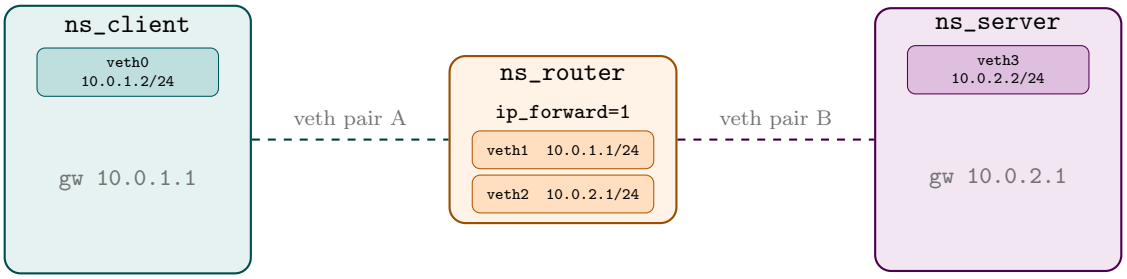
Hands-On #11

Rete virtuale in C: Netlink, namespace e routing

Last Update: 16 aprile 2026

Scenario

In questo ultimo HO della parte del corso dedicata alle reti costruiremo da zero — **in C puro, senza chiamare mai `system()` o `exec()`** — la seguente topologia:



Il codice userà esclusivamente le API del kernel Linux: **Netlink**, `unshare()`, `setns()`, `mount()`. Queste sono le stesse primitive che usa il comando `ip` internamente.

Mappa concettuale: come parla il kernel con sé stesso

Prima di leggere il codice ripassiamo tre concetti.

Network namespace

Un *network namespace* è una copia isolata dello stack di rete del kernel: interfacce, tabelle di routing, socket in ascolto, regole iptables. Il namespace di default (*root namespace*) esiste da sempre. Creare un namespace nuovo significa dire al kernel: "tieni una seconda copia di tutto questo, invisibile al resto".

Socket Netlink

Netlink è il canale di comunicazione tra lo spazio utente e il kernel per le operazioni di rete (e non solo). Non usa porte, non usa TCP: è un tipo di socket speciale (`AF_NETLINK`) su cui si mandano messaggi strutturati.

Il sotto-tipo che useremo è `NETLINK_ROUTE`, che gestisce interfacce, indirizzi IP e tabelle di routing. Ogni messaggio ha questa struttura:

nlmsghdr	Header specifico	Attributi (rtattr)
tipo, flag, seq	ifinfomsg / rtmsg / ...	chiave=valore

Gli attributi (`rtattr`) sono coppie tipo/valore concatenate in memoria. La macro `RTA_DATA(rta)` restituisce il puntatore al valore; `RTA_LENGTH(n)` calcola la dimensione totale allineata.

Coppia veth

Una coppia `veth` è un cavo virtuale bidirezionale: i byte che entrano da un'estremità escono dall'altra. Si crea sempre in coppia; i due capi si spostano poi in namespace diversi. Questo è esattamente ciò che fa Docker per collegare ogni container alla rete.

Il routing in Linux

Non una, ma tre tabelle

Linux non ha una sola tabella di routing: ne ha **tre per default**, ognuna identificata da un numero e un nome.

Numero	Nome	Scopo
255	`local`	Indirizzi locali e broadcast — gestita solo dal kernel
254	`main`	La tabella "normale", quella che vedi con `ip route show`
253	`default`	Fallback, quasi sempre vuota

Per ogni pacchetto il kernel le consulta nell'ordine stabilito dalle policy, fermandosi alla prima corrispondenza trovata.

La tabella `local`: il routing verso se stesso

Il kernel la popola **automaticamente** ogni volta che assegni un indirizzo IP a un'interfaccia con `nl_set_addr()`. Tu non la tocchi mai direttamente. Dopo aver configurato `ns_router` in `topo.c`, la tabella `local` del router contiene:

```
# ip netns exec ns_router ip route show table local
local 10.0.1.1 dev veth1 proto kernel scope host src 10.0.1.1
broadcast 10.0.1.0 dev veth1 proto kernel scope link src 10.0.1.1
broadcast 10.0.1.255 dev veth1 proto kernel scope link src 10.0.1.1
local 10.0.2.1 dev veth2 proto kernel scope host src 10.0.2.1
broadcast 10.0.2.0 dev veth2 proto kernel scope link src 10.0.2.1
broadcast 10.0.2.255 dev veth2 proto kernel scope link src 10.0.2.1
local 127.0.0.1 dev lo proto kernel scope host src 127.0.0.1
```

Il campo `scope host` significa: "questo indirizzo IP sono io". Il pacchetto non esce da nessuna interfaccia — viene consegnato direttamente allo stack TCP/IP locale. È per questo che `ping 10.0.1.1` dal router stesso funziona istantaneamente, senza percorrere nessun cavo.

La tabella `main`: il routing verso gli altri

È la tabella che manipoliamo con `nl_add_route()` in `topo.c`. Contiene due tipi di entry.

Route "connected" — aggiunte automaticamente dal kernel quando porti un'interfaccia UP con un indirizzo assegnato, cioè dopo `nl_set_addr()` + `nl_set_up()`. Nel router:

```
10.0.1.0/24 dev veth1 proto kernel scope link src 10.0.1.1
10.0.2.0/24 dev veth2 proto kernel scope link src 10.0.2.1
```

Queste due righe spiegano perché il router non ha bisogno di route statiche: conosce già entrambe le subnet perché ha un'interfaccia dentro ciascuna.

Route statiche — aggiunte manualmente da `nl_add_route()`, come le default route di client e server:

```
# tabella main di ns_client
default via 10.0.1.1 dev veth0              <- aggiunta da nl_add_route()
10.0.1.0/24 dev veth0 proto kernel ...      <- aggiunta dal kernel (
    connected)
```

Le policy: in quale ordine vengono consultate le tabelle

L'ordine è definito da regole di policy, visibili con `ip rule show`:

```
0:      from all lookup local      <- prima di tutto, sempre
32766:  from all lookup main       <- poi
32767:  from all lookup default    <- infine
```

Il numero a sinistra è la priorità (più basso = prima). Il kernel scorre le regole dall'alto verso il basso e si ferma al primo match. La regola `0` garantisce che la tabella `local` sia sempre il primo controllo: se il pacchetto è diretto a un indirizzo locale, viene consegnato allo stack senza nemmeno consultare `main`.

Il percorso completo di un pacchetto dal client al server nella topologia di questo HO è quindi:

```
ns_client deve inviare a 10.0.2.2
1.  consulta local:  10.0.2.2 non è un mio indirizzo → non trovato
2.  consulta main:  trova "default via 10.0.1.1 dev veth0" → match
3.  manda il pacchetto a 10.0.1.1 su veth0

ns_router riceve il pacchetto su veth1
1.  consulta local:  10.0.2.2 non è un mio indirizzo → non trovato
2.  ip_forward = 1:  non scartare, instrada
3.  consulta main:  trova "10.0.2.0/24 dev veth2" → match
4.  manda il pacchetto su veth2 verso 10.0.2.2

ns_server riceve il pacchetto su veth3
1.  consulta local:  10.0.2.2 sono io!  → consegna allo stack locale
```

IP forwarding: l'interruttore del router

Per default Linux **scarta** qualsiasi pacchetto la cui destinazione non è nella tabella `local` del namespace corrente — comportamento corretto per un host terminale. Abilitare `ip_forward` cambia questo: il kernel, invece di scartare, consulta `main` e instrada il pacchetto verso il next-hop corretto.

`ip_forward` non è un'opzione Netlink: è un parametro **sysctl**, configurato scrivendo nel filesystem virtuale `/proc`. In `topo.c` la funzione `nl_ip_forward()` fa esattamente questo:

Listing 1: nl_ip_forward() in topo.c

```c
static void nl_ip_forward(void) {
    /* /proc/sys/net/ipv4/ip_forward e' un file virtuale:
     * scrivere "1\n" abilita il forwarding nel namespace corrente.
     * Ogni namespace ha il proprio file, indipendente dagli altri. */
    int fd = open("/proc/sys/net/ipv4/ip_forward", O_WRONLY);
    if (fd < 0) { perror("open ip_forward"); return; }
    write(fd, "1\n", 2);
    close(fd);
}
```

Due punti critici da ricordare: questa funzione va chiamata *dopo* `netns_enter(NS_ROUTER)`, perché ogni namespace ha il proprio `ip_forward` indipendente. E abilitarlo non basta da solo: senza le route corrette in `main`, i pacchetti vengono scartati comunque con un ICMP *Network Unreachable*.

Struttura del programma

Il programma `topo.c` è organizzato in tre livelli:

```
Livello 3 - main():     orchestrazione della topologia
Livello 2 - netns_*():   gestione dei namespace
(unshare/setns/mount)
Livello 1 - nl_*():      operazioni Netlink di basso livello
```

Il codice, funzione per funzione

`nl_open()` — Apre il socket Netlink

Listing 2: `nl_open()`: apertura e bind del socket Netlink

```c
static int nl_open(void) {
    /* AF_NETLINK = famiglia Netlink (non AF_INET!)
     * SOCK_RAW   = accesso grezzo ai messaggi Netlink
     * NETLINK_ROUTE = sotto-tipo per interfacce/routing */
    int fd = socket(AF_NETLINK, SOCK_RAW | SOCK_CLOEXEC, NETLINK_ROUTE);
    if (fd < 0) { perror("socket(NETLINK_ROUTE)"); exit(1); }

    struct sockaddr_nl sa = {
        .nl_family = AF_NETLINK,
        .nl_pid    = getpid(),  /* identifica il processo mittente */
    };
    /* bind() e' necessario per ricevere le risposte del kernel */
    if (bind(fd, (struct sockaddr *)&sa, sizeof(sa)) < 0) {
        perror("bind(netlink)"); exit(1);
    }
    return fd;
}
```

Il campo `nl_pid` nel bind identifica univocamente il processo mittente. Le risposte del kernel tornano allo stesso pid.

`nl_send()` — Invia un messaggio e legge la risposta

Listing 3: `nl_send()`: invio e verifica ACK

```c
static int nl_send(int fd, struct nlmsghdr *nlh, const char *op) {
    struct iovec iov  = { nlh, nlh->nlmsg_len };
    struct msghdr msg = { .msg_iov = &iov, .msg_iovlen = 1 };
    if (sendmsg(fd, &msg, 0) < 0) { /* ...errore... */ return -1; }

    char buf[NL_BUFSIZE];
    ssize_t n = recv(fd, buf, sizeof(buf), 0);
    /* ...controllo n... */

    struct nlmsghdr *rep = (struct nlmsghdr *)buf;
    if (rep->nlmsg_type == NLMSG_ERROR) {
        struct nlmsgerr *err = (struct nlmsgerr *)NLMSG_DATA(rep);
        /* err->error == 0 significa ACK (successo).
         * err->error < 0 significa errore; il valore e' -errno */
        if (err->error != 0) {
            fprintf(stderr, "[%s] Netlink error: %s\n",
                    op, strerror(-err->error));
            return -1;
        }
    }
    return 0;
}
```

Ogni messaggio inviato con il flag `NLM_F_ACK` riceve una risposta `NLMSG_ERROR`: se `err->error == 0` è un ACK di successo, altrimenti contiene `-errno`.

nl_addattr() — Aggiunge un attributo al messaggio

Listing 4: nl_addattr(): scrittura di un rtattr in coda

```c
static struct rtattr *nl_addattr(struct nlmsghdr *nlh, int type,
                                 const void *data, int len) {
    /* RTA_LENGTH(n) = sizeof(rtattr) + n, allineato a 4 byte */
    int alen = RTA_LENGTH(len);

    /* L'attributo viene scritto subito dopo la fine del messaggio.
     * NLMSG_ALIGN garantisce che l'offset sia allineato. */
    struct rtattr *rta = (struct rtattr *)
        ((char *)nlh + NLMSG_ALIGN(nlh->nlmsg_len));
    rta->rta_type = type;
    rta->rta_len  = alen;
    if (data) memcpy(RTA_DATA(rta), data, len);

    /* Aggiorna la lunghezza del messaggio */
    nlh->nlmsg_len = NLMSG_ALIGN(nlh->nlmsg_len) + RTA_ALIGN(alen);
    return rta;
}
```

Tutti i messaggi Netlink sono buffer contigui in memoria. Aggiungere un attributo significa: calcolare dove finisce il messaggio corrente, scriverci sopra l'header `rtattr` + il valore, e aggiornare `nlmsg_len`.

netns_create() — Crea un named network namespace

Questa è la funzione più delicata. Il meccanismo in quattro passi:

Listing 5: netns_create(): creazione e ancoraggio al filesystem

```c
static void netns_create(const char *path) {
    /* Passo 1: crea il file che fara' da "mount point".
     * Senza di esso mount() non ha su cosa agganciarsi. */
    int fd = open(path, O_RDONLY | O_CREAT | O_EXCL, 0);
    if (fd < 0 && errno != EEXIST) { /* ...errore... */ exit(1); }
    if (fd >= 0) close(fd);

    /* Passo 2: entra in un nuovo network namespace.
     * CLONE_NEWNET crea un namespace vuoto e ci sposta il processo.
     * Da questo momento /proc/self/ns/net punta al nuovo namespace. */
    if (unshare(CLONE_NEWNET) < 0) { perror("unshare"); exit(1); }

    /* Passo 3: bind-mount.
     * Monta /proc/self/ns/net (il nuovo ns) sopra il file creato.
     * Questo "ancora" il namespace al filesystem: sopravvivera'
     * anche quando nessun processo ci vive piu' dentro. */
    if (mount("/proc/self/ns/net", path, "none", MS_BIND, NULL) < 0) {
        fprintf(stderr, "mount(%s): %s\n", path, strerror(errno));
        exit(1);
    }

    /* Passo 4: torna al root namespace.
     * /proc/1/ns/net e' sempre il network namespace del processo init,
     * cioe' il root namespace. setns() ci rientra. */
    int root_ns = open("/proc/1/ns/net", O_RDONLY);
    if (setns(root_ns, CLONE_NEWNET) < 0) { perror("setns"); exit(1); }
    close(root_ns);
}
```

> ▷ **Perché il bind-mount?**
>
> Un network namespace esiste finché esiste almeno un riferimento ad esso: un processo che ci vive, oppure un file bind-mounted. Senza il bind-mount, il namespace verrebbe distrutto non appena `netns_create()` ritorna (perché nessun processo ci rimane dentro). Il bind-mount in `/run/netns/` è esattamente quello che fa il comando `ip netns add` internamente.

`nl_create_veth()` — Crea una coppia veth

Listing 6: `nl_create_veth()`: messaggio RTM_NEWLINK per veth

```
static void nl_create_veth(int nlfd, const char *name,
                           const char *peer_name) {
    /* RTM_NEWLINK con NLM_F_CREATE: "crea una nuova interfaccia"
     * NLM_F_EXCL: fallisce se esiste gia' */
    nlh->nlmsg_type  = RTM_NEWLINK;
    nlh->nlmsg_flags = NLM_F_REQUEST | NLM_F_CREATE | NLM_F_EXCL |
        NLM_F_ACK;

    /* Nome dell'interfaccia principale (es. "veth0") */
    nl_addattr(nlh, IFLA_IFNAME, name, strlen(name)+1);

    /* IFLA_LINKINFO contiene il tipo del link e i suoi parametri */
    struct rtattr *linkinfo = nl_nest_open(nlh, IFLA_LINKINFO);
    nl_addattr(nlh, IFLA_INFO_KIND, "veth", 5); /* tipo = veth */

    /* IFLA_INFO_DATA: parametri specifici del tipo veth */
    struct rtattr *data = nl_nest_open(nlh, IFLA_INFO_DATA);

    /* VETH_INFO_PEER: descrizione del secondo capo del cavo.
     * Contiene un ifinfomsg completo + IFLA_IFNAME del peer.
     * Il kernel crea entrambe le interfacce in un'unica operazione. */
    struct rtattr *peer_rta = nl_nest_open(nlh, VETH_INFO_PEER);
    struct ifinfomsg *peer_ifi = (struct ifinfomsg *)
        ((char *)nlh + NLMSG_ALIGN(nlh->nlmsg_len));
    memset(peer_ifi, 0, sizeof(*peer_ifi));
    nlh->nlmsg_len += sizeof(struct ifinfomsg);
    nl_addattr(nlh, IFLA_IFNAME, peer_name, strlen(peer_name)+1);
    nl_nest_close(nlh, peer_rta);

    nl_nest_close(nlh, data);
    nl_nest_close(nlh, linkinfo);
    nl_send(nlfd, nlh, "create veth");
}
```

La struttura ad attributi annidati rispecchia la gerarchia logica: il link ha un tipo (`IFLA_INFO_KIND`), il tipo ha dei parametri (`IFLA_INFO_DATA`), i parametri descrivono il peer (`VETH_INFO_PEER`). È come XML, ma in binario e in memoria.

`nl_move_to_ns()` — Sposta un'interfaccia in un namespace

Listing 7: `nl_move_to_ns()`: RTM_NEWLINK con IFLA_NET_NS_FD

```
static void nl_move_to_ns(int nlfd, const char *ifname,
                          const char *ns_path) {
    /* Recupera l'ifindex: identificatore numerico dell'interfaccia.
     * Il kernel usa l'ifindex, non il nome, come chiave primaria. */
    int ifindex = nl_iface_index(ifname);

    /* Apre il namespace di destinazione come file descriptor.
```

```
     * Il kernel accetta IFLA_NET_NS_FD: "sposta questa interfaccia
     * nel namespace referenziato da questo fd". */
    int ns_fd = open(ns_path, O_RDONLY);

    /* RTM_NEWLINK sull'interfaccia esistente (non NLM_F_CREATE):
     * "modifica" l'interfaccia ifindex aggiungendole l'attributo
     * IFLA_NET_NS_FD -> spostamento nel namespace. */
    ifi->ifi_index = ifindex;
    nl_addattr(nlh, IFLA_NET_NS_FD, &ns_fd, sizeof(ns_fd));
    nl_send(nlfd, nlh, "move to ns");
    close(ns_fd);
}
```

nl_set_addr() — Assegna un indirizzo IP

Listing 8: nl_set_addr(): RTM_NEWADDR

```
static void nl_set_addr(int nlfd, const char *ifname,
                        const char *ip, int prefix) {
    struct in_addr addr;
    inet_pton(AF_INET, ip, &addr); /* stringa -> 32 bit network order */

    /* RTM_NEWADDR: aggiunge un indirizzo a un'interfaccia.
     * ifa_prefixlen: lunghezza maschera (es. 24 per /24).
     * ifa_scope: RT_SCOPE_UNIVERSE = indirizzo globale raggiungibile. */
    ifa->ifa_family    = AF_INET;
    ifa->ifa_prefixlen = prefix;
    ifa->ifa_scope     = RT_SCOPE_UNIVERSE;
    ifa->ifa_index     = ifindex;

    /* IFA_LOCAL e IFA_ADDRESS: per IPv4 sono identici.
     * In IPv6 IFA_ADDRESS e' l'anycast, IFA_LOCAL il locale. */
    nl_addattr(nlh, IFA_LOCAL,   &addr, sizeof(addr));
    nl_addattr(nlh, IFA_ADDRESS, &addr, sizeof(addr));
    nl_send(nlfd, nlh, "set addr");
}
```

nl_set_up() — Porta l'interfaccia in stato UP

Listing 9: nl_set_up(): RTM_NEWLINK con IFF_UP

```
static void nl_set_up(int nlfd, const char *ifname) {
    /* RTM_NEWLINK senza NLM_F_CREATE = modifica un'interfaccia esistente.
     *
     * ifi_flags:  i flag da impostare (IFF_UP = attiva l'interfaccia)
     * ifi_change: maschera che indica quali bit di ifi_flags considerare.
     *
     * Senza ifi_change = IFF_UP il kernel ignorerebbe la modifica:
     * il campo flags ha 16 bit, ifi_change dice "guarda solo questi". */
    ifi->ifi_flags  = IFF_UP;
    ifi->ifi_change = IFF_UP;
    nl_send(nlfd, nlh, "set UP");
}
```

nl_add_route() — Aggiunge una route

Listing 10: nl_add_route(): RTM_NEWROUTE

```
static void nl_add_route(int nlfd, const char *dst_ip, int dst_prefix,
                         const char *gw_ip, const char *ifname) {
    /* RTM_NEWROUTE: aggiunge una entry nella tabella di routing.
     *
```

```
     * rtm_dst_len = 0 e dst_ip = NULL -> default route (0.0.0.0/0)
     * rtm_table   = RT_TABLE_MAIN     -> tabella principale
     * rtm_scope   = RT_SCOPE_UNIVERSE -> next hop fuori dalla subnet
     * rtm_type    = RTN_UNICAST       -> traffico unicast normale */
    rtm->rtm_dst_len  = dst_prefix;
    rtm->rtm_table    = RT_TABLE_MAIN;
    rtm->rtm_protocol = RTPROT_STATIC; /* aggiunta manualmente */
    rtm->rtm_scope    = RT_SCOPE_UNIVERSE;
    rtm->rtm_type     = RTN_UNICAST;

    if (dst_ip) {
        struct in_addr dst; inet_pton(AF_INET, dst_ip, &dst);
        nl_addattr(nlh, RTA_DST, &dst, sizeof(dst));
    }
    struct in_addr gw; inet_pton(AF_INET, gw_ip, &gw);
    nl_addattr(nlh, RTA_GATEWAY, &gw, sizeof(gw)); /* next hop */
    nl_addattr(nlh, RTA_OIF, &ifindex, sizeof(ifindex)); /* iface uscita
        */
    nl_send(nlfd, nlh, "add route");
}
```

main() — Orchestrazione

Listing 11: main(): i 6 passi di configurazione della topologia

```
int main(void) {
    mkdir(NETNS_RUN_DIR, 0755); /* assicura che /run/netns esista */

    /* 1. Crea i tre namespace (bind-mounted in /run/netns/) */
    netns_create(NS_CLIENT);
    netns_create(NS_ROUTER);
    netns_create(NS_SERVER);

    /* 2. Nel root namespace: crea le due coppie veth.
     *    Tutte e quattro le interfacce partono nel root namespace. */
    int nlfd = nl_open();
    nl_create_veth(nlfd, "veth0", "veth1");
    nl_create_veth(nlfd, "veth2", "veth3");

    /* 3. Sposta ogni interfaccia nel namespace di destinazione.
     *    Dopo questi comandi veth0..3 non sono piu' visibili
     *    nel root namespace. */
    nl_move_to_ns(nlfd, "veth0", NS_CLIENT);
    nl_move_to_ns(nlfd, "veth1", NS_ROUTER);
    nl_move_to_ns(nlfd, "veth2", NS_ROUTER);
    nl_move_to_ns(nlfd, "veth3", NS_SERVER);
    close(nlfd);

    /* 4. Configura ns_client: entra, configura, esci. */
    int root_fd = netns_enter(NS_CLIENT);
    nlfd = nl_open();                          /* nuovo socket nel ns! */
    nl_set_addr(nlfd, "veth0", "10.0.1.2", 24);
    nl_set_up(nlfd,   "veth0");
    nl_set_up(nlfd,   "lo");
    nl_add_route(nlfd, NULL, 0, "10.0.1.1", "veth0"); /* default gw */
    close(nlfd);
    netns_restore(root_fd);

    /* 5. Configura ns_router */
    root_fd = netns_enter(NS_ROUTER);
    nlfd = nl_open();
    nl_set_addr(nlfd, "veth1", "10.0.1.1", 24);
    nl_set_addr(nlfd, "veth2", "10.0.2.1", 24);
```

```c
    nl_set_up(nlfd, "veth1"); nl_set_up(nlfd, "veth2"); nl_set_up(nlfd, "lo");
    nl_ip_forward(); /* scrive 1 in /proc/sys/net/ipv4/ip_forward */
    close(nlfd);
    netns_restore(root_fd);

    /* 6. Configura ns_server */
    root_fd = netns_enter(NS_SERVER);
    nlfd = nl_open();
    nl_set_addr(nlfd, "veth3", "10.0.2.2", 24);
    nl_set_up(nlfd, "veth3"); nl_set_up(nlfd, "lo");
    nl_add_route(nlfd, NULL, 0, "10.0.2.1", "veth3");
    close(nlfd);
    netns_restore(root_fd);

    return 0;
}
```

▷ Perché aprire un nuovo socket Netlink dopo `netns_enter()`?

Un socket Netlink è legato al network namespace in cui viene creato. Se crei il socket nel root namespace e poi entri in `ns_client`, il socket vede ancora le interfacce del root namespace — non quelle di `ns_client`. Bisogna aprire un nuovo `nl_open()` *dopo* `netns_enter()`.

Compilazione ed esecuzione

```
gcc -Wall -Wextra -o topo topo.c
sudo ./topo
```

Output atteso:

```
=== Creazione namespace ===
  [OK] namespace creato: /run/netns/ns_client
  [OK] namespace creato: /run/netns/ns_router
  [OK] namespace creato: /run/netns/ns_server

=== Creazione coppie veth (root ns) ===
  [OK] create veth veth0<->veth1
  [OK] create veth veth2<->veth3

=== Spostamento interfacce ===
  [OK] move veth0 -> /run/netns/ns_client
  [OK] move veth1 -> /run/netns/ns_router
  [OK] move veth2 -> /run/netns/ns_router
  [OK] move veth3 -> /run/netns/ns_server

=== Configurazione ns_client ===
  [OK] addr 10.0.1.2/24 on veth0
  ...
=== Topologia pronta ===
```

Verifica la connettività:

```
# Ping client -> server (attraversa il router: 2 hop)
ip netns exec ns_client ping -c 3 10.0.2.2

# Tabella di routing del client (deve mostrare la default route)
ip netns exec ns_client ip route show

# Tabella di routing del router (deve mostrare entrambe le subnet)
ip netns exec ns_router ip route show
```

Pulizia (rimuove namespace e interfacce):

```
ip netns del ns_client; ip netns del ns_router; ip netns del ns_server
```